

Random Forest

Kachun Huang

2025-04-02

Random Forest in R

Import Dependencies

```
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'  
  
## The following objects are masked from 'package:stats':  
##  
##     filter, lag  
  
## The following objects are masked from 'package:base':  
##  
##     intersect, setdiff, setequal, union
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3  
  
## Loading required package: ggplot2  
  
## Loading required package: lattice  
  
## Warning: package 'lattice' was built under R version 4.3.3
```

```
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.3.3  
  
## randomForest 4.7-1.2  
  
## Type rfNews() to see new features/changes/bug fixes.
```

```
##
## Attaching package: 'randomForest'

## The following object is masked from 'package:ggplot2':
##
##     margin

## The following object is masked from 'package:dplyr':
##
##     combine
```

Fit Random Forest Model

```
model <- randomForest(
  formula,
  data,
  importance = TRUE
)
```

Parameter:

importance : a matrix with **nclass** + 2 (for classification) or two (for regression) columns. For classification, the first **nclass** columns are the class-specific measures computed as mean decrease in accuracy. The **nclass** + 1st column is the mean decrease in accuracy over all classes. The last column is the mean decrease in Gini index. For Regression, the first column is the mean decrease in accuracy and the second the mean decrease in MSE. If **importance=FALSE**, the last measure is still returned as a vector.

Output Model

```
print(model)
```

Prediction

```
pred_rf<-predict(
  model,
  test_data
)
```

Variable Importance

```
varImpPlot(model)
```

The plot ranks features based on their **importance scores**, which measure how much each feature contributes to the Random Forest's predictive performance. Features with higher importance scores are more influential in making accurate predictions, while those with lower scores have less impact.

- **X-axis:** Typically shows the importance score (or a scaled version of it).
- **Y-axis:** Lists the features, often sorted in descending order of importance.
- The plot visually highlights which variables matter most, making it easier to interpret the model and focus on key drivers.

Random Forests calculate feature importance in a few different ways. The two most common methods are:

a. Mean Decrease in Impurity (Gini Importance)

- **What it measures:** This method calculates the total reduction in impurity (e.g., Gini impurity for classification or variance for regression) brought by a feature across all trees in the forest.
- **How it works:**
 - For each tree, the algorithm evaluates how much a feature reduces the impurity when it's used to split a node.
 - The importance of a feature is the average of these reductions across all trees.
 - Features that consistently reduce impurity by a large amount (e.g., by creating purer child nodes) get higher scores.
- **Use case:** Common in classification tasks where Gini impurity or entropy is used.

b. Mean Decrease in Accuracy (Permutation Importance)

- **What it measures:** This method assesses how much the model's accuracy decreases when a feature's values are randomly shuffled, breaking its relationship with the target variable.
- **How it works:**
 - The model's performance (e.g., accuracy, RMSE) is calculated on a validation set or out-of-bag (OOB) samples.
 - For each feature, its values are permuted, and the model's performance is recalculated.
 - The drop in performance after permutation indicates the feature's importance: a larger drop means the feature was more important.
- **Use case:** Often considered more robust because it directly measures the impact on predictive performance, not just node purity.

Hyper-parameter Tuning

In caret package, we can do hyper-parameter tuning and find the best tune.

```
# 10-fold CV
control <- trainControl(
  method = "cv",
  number = 10
)

# train the rf using 10-fold CV
rf_caret <- train(
```

```

formula,
data,
method = "rf",
trControl = control,
importance = TRUE
)

```

Output the bestTune

```
print(rf_caret$bestTune)
```

Classification Example

Import data

```
bank <- read.csv("/Users/thomashuang/Downloads/banknote.csv")
```

```
str(bank)
```

```

## 'data.frame': 1372 obs. of 5 variables:
## $ variance: num 3.622 4.546 3.866 3.457 0.329 ...
## $ skewness: num 8.67 8.17 -2.64 9.52 -4.46 ...
## $ kurtosis: num -2.81 -2.46 1.92 -4.01 4.57 ...
## $ entropy : num -0.447 -1.462 0.106 -3.594 -0.989 ...
## $ class : int 1 1 1 1 1 1 1 1 1 1 ...

```

Remove Missing Value

```

bank <- na.omit(bank)
str(bank)

```

```

## 'data.frame': 1372 obs. of 5 variables:
## $ variance: num 3.622 4.546 3.866 3.457 0.329 ...
## $ skewness: num 8.67 8.17 -2.64 9.52 -4.46 ...
## $ kurtosis: num -2.81 -2.46 1.92 -4.01 4.57 ...
## $ entropy : num -0.447 -1.462 0.106 -3.594 -0.989 ...
## $ class : int 1 1 1 1 1 1 1 1 1 1 ...

```

There are no missing value in the dataset

Factorize Response

```
bank$class <- as.factor(bank$class)
```

Data Splitting

```
set.seed(123)
training.samples <- bank$class %>% createDataPartition(p=0.75, list=FALSE)
train.data <- bank[training.samples,]
test.data <- bank[-training.samples,]
```

Random Forest Model

```
rf_model <- randomForest(
  class~.,
  data = train.data,
  importance = TRUE
)
print(rf_model)

##
## Call:
## randomForest(formula = class ~ ., data = train.data, importance = TRUE)
##              Type of random forest: classification
##              Number of trees: 500
## No. of variables tried at each split: 2
##
##              OOB estimate of  error rate: 0.68%
## Confusion matrix:
##      0   1 class.error
## 0 456   2 0.004366812
## 1   5 567 0.008741259
```

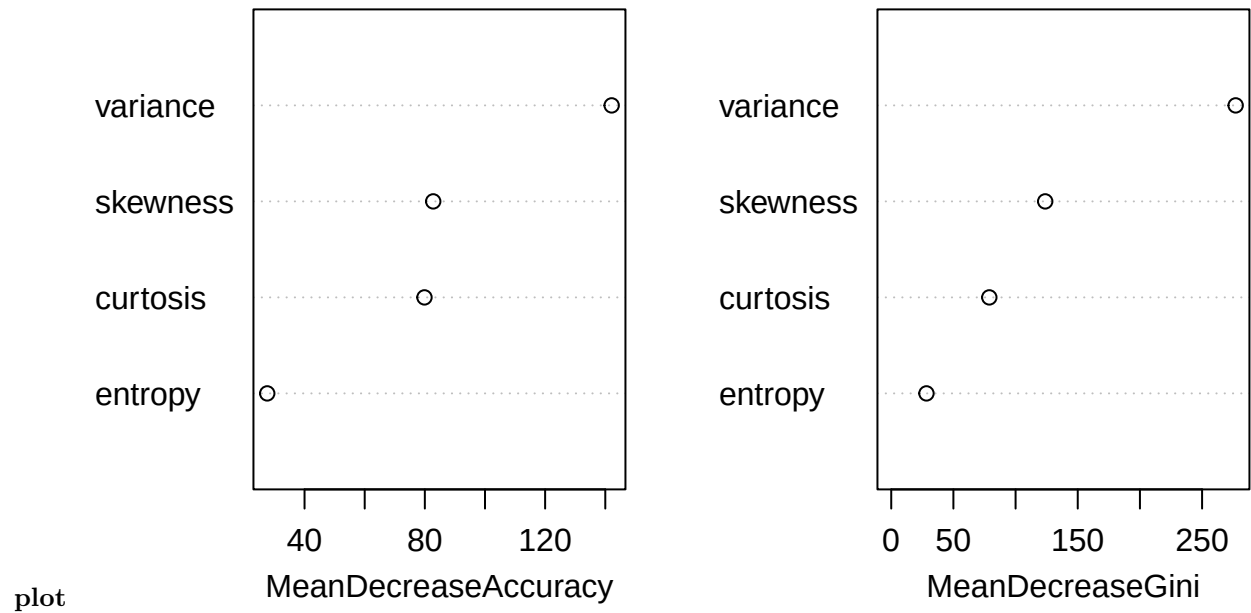
Variable Importance (Both)

```
importance(rf_model)
```

	0	1	MeanDecreaseAccuracy	MeanDecreaseGini
variance	132.60928	102.42340	142.12543	276.99668
skewness	84.66696	52.26393	82.77901	123.82411
curtosis	64.97725	51.53346	79.87335	78.87259
entropy	22.27784	18.24094	27.58992	28.34306

```
varImpPlot(rf_model)
```

rf_model

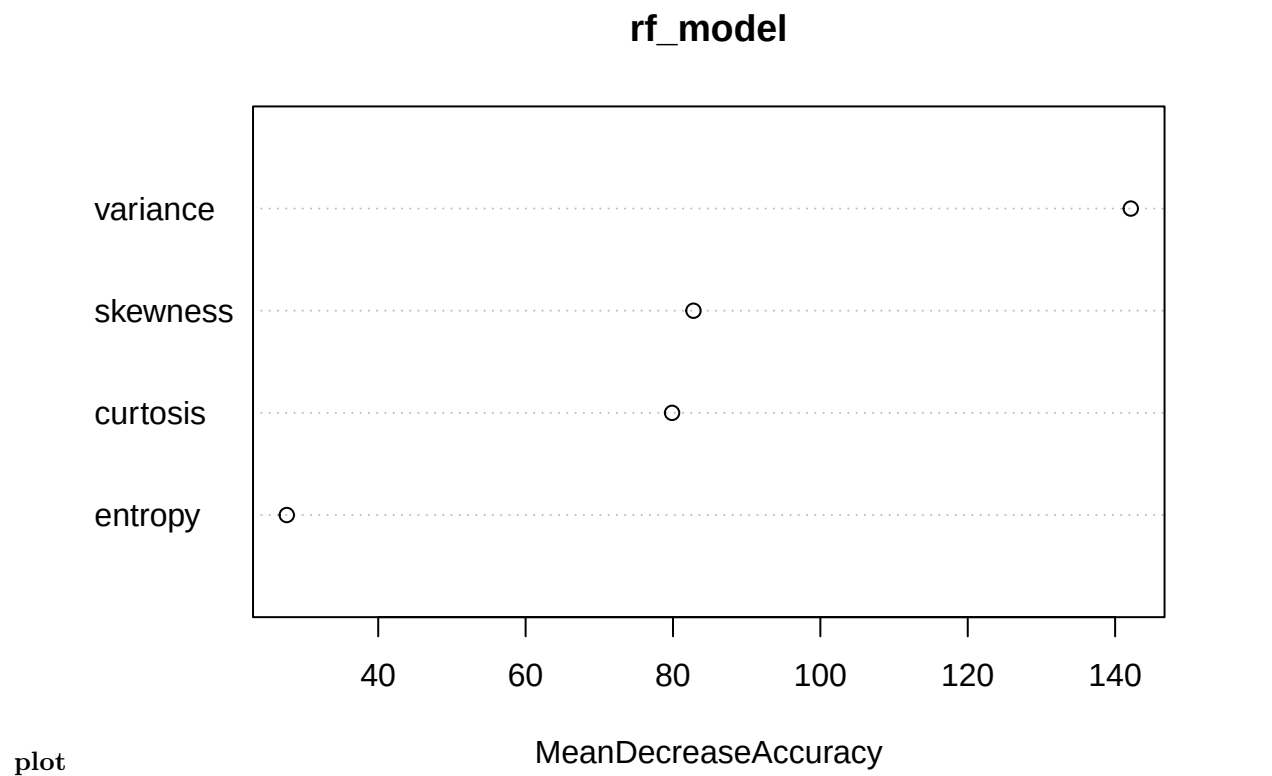


Mean Decrease Accuracy

```
varImp(rf_model, type=1)
```

```
##           Overall
## variance 142.12543
## skewness  82.77901
## curtosis  79.87335
## entropy   27.58992
```

```
# Plot MeanDecreaseAccuracy
varImpPlot(rf_model, type=1)
```



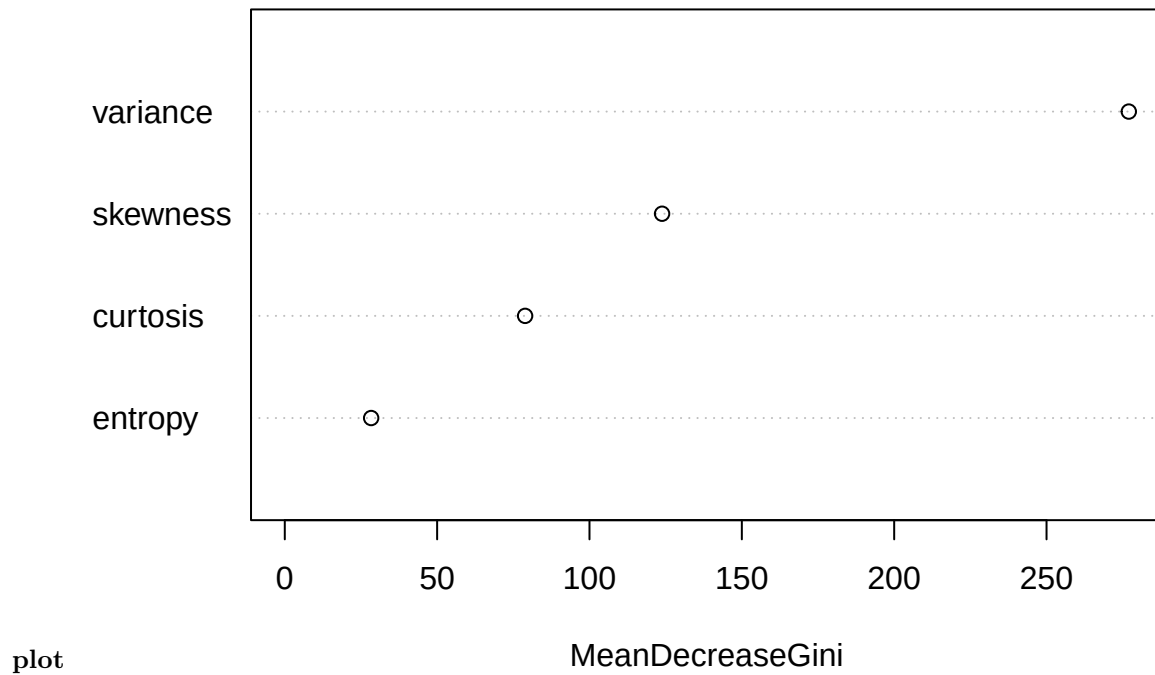
Mean Decrease Gini

```
varImp(rf_model, type=2)
```

```
##           Overall
## variance 276.99668
## skewness 123.82411
## curtosis  78.87259
## entropy  28.34306
```

```
# Plot MeanDecreaseGini
varImpPlot(rf_model, type=2)
```

rf_model



Prediction

```
pred_rf <- predict(rf_model, test.data)
```

Confusion Matrix

```
confusionMatrix(pred_rf, test.data$class, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 148    1
##           1   4 189
##
##           Accuracy : 0.9854
##           95% CI : (0.9662, 0.9952)
##           No Information Rate : 0.5556
##           P-Value [Acc > NIR] : <2e-16
##
##           Kappa : 0.9703
##
##           McNemar's Test P-Value : 0.3711
##
##           Sensitivity : 0.9947
```

```
##           Specificity : 0.9737
##           Pos Pred Value : 0.9793
##           Neg Pred Value : 0.9933
##           Prevalence : 0.5556
##           Detection Rate : 0.5526
##           Detection Prevalence : 0.5643
##           Balanced Accuracy : 0.9842
##
##           'Positive' Class : 1
##
```

Hyper-parameter Tuning

```
set.seed(123)
# 10-fold CV
control <- trainControl(
  method = "cv",
  number = 10
)

# train the rf using 10-fold CV
rf_caret <- train(
  class ~ .,
  data = train.data,
  method = "rf",
  trControl = control,
  importance = TRUE
)

print(rf_caret)
```

```
## Random Forest
##
## 1030 samples
## 4 predictor
## 2 classes: '0', '1'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 927, 927, 927, 927, 927, 927, ...
## Resampling results across tuning parameters:
##
##  mtry  Accuracy  Kappa
##  2     0.9922330 0.9842950
##  3     0.9932039 0.9862385
##  4     0.9902722 0.9802915
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was mtry = 3.
```

Access bestTune

```
print(rf_caret$bestTune)
```

```
## mtry  
## 2 3
```

Access final Model

```
final_model <- rf_caret$finalModel  
final_model
```

```
##  
## Call:  
## randomForest(x = x, y = y, mtry = param$mtry, importance = TRUE)  
## Type of random forest: classification  
## Number of trees: 500  
## No. of variables tried at each split: 3  
##  
## OOB estimate of error rate: 0.58%  
## Confusion matrix:  
## 0 1 class.error  
## 0 456 2 0.004366812  
## 1 4 568 0.006993007
```

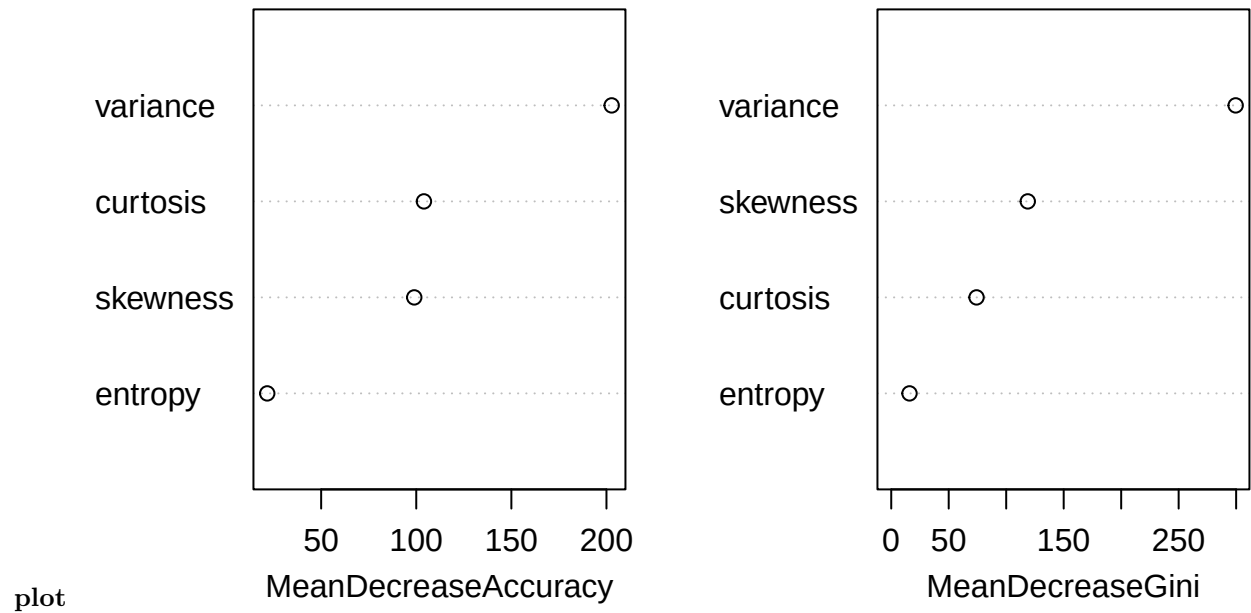
Variable Importance (Both)

```
importance(final_model)
```

```
##          0          1 MeanDecreaseAccuracy MeanDecreaseGini  
## variance 215.45782 122.75948          202.73967          299.58446  
## skewness 110.60161  53.05652           98.94061          118.75781  
## curtosis  85.59472  62.79343          104.02693           74.21030  
## entropy   18.18065  15.79927           21.64304           15.89303
```

```
varImpPlot(final_model)
```

final_model

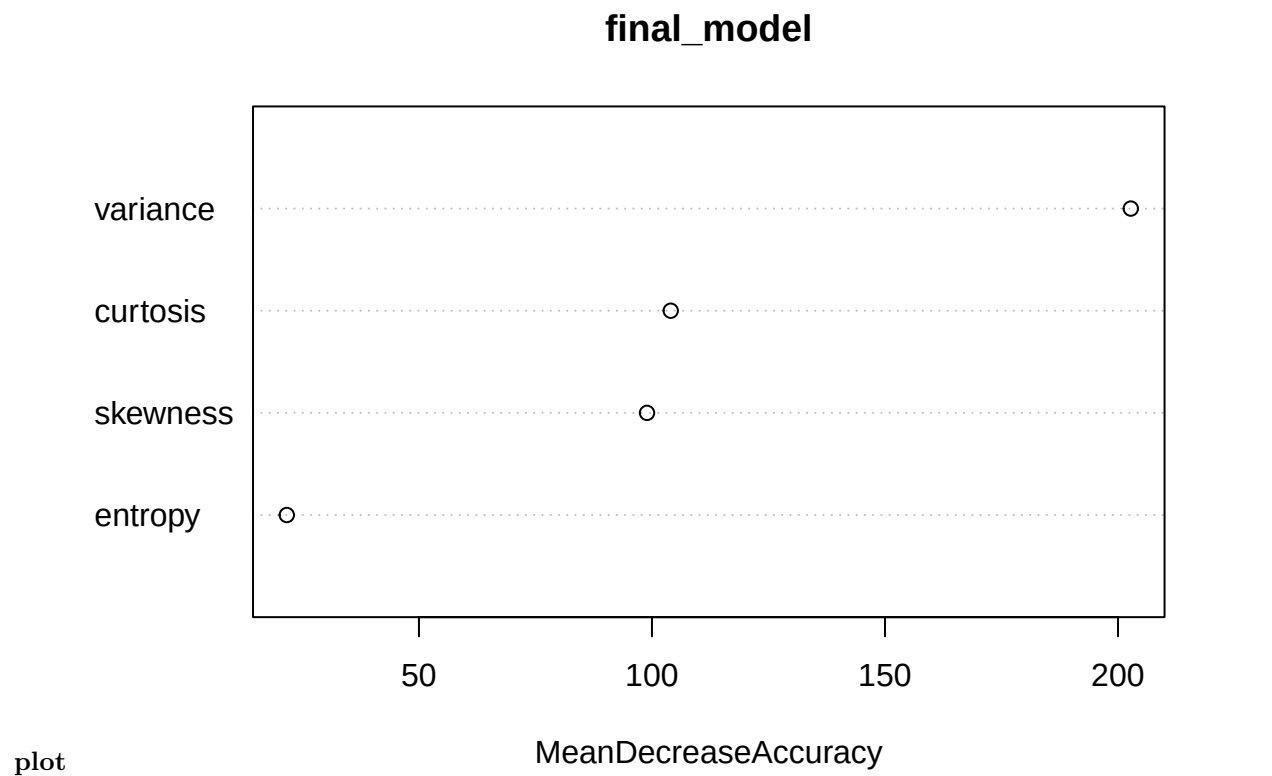


Mean Decrease Accuracy

```
varImp(final_model, type=1)
```

```
##           Overall
## variance 202.73967
## skewness  98.94061
## curtosis 104.02693
## entropy   21.64304
```

```
# Plot MeanDecreaseAccuracy
varImpPlot(final_model, type=1)
```



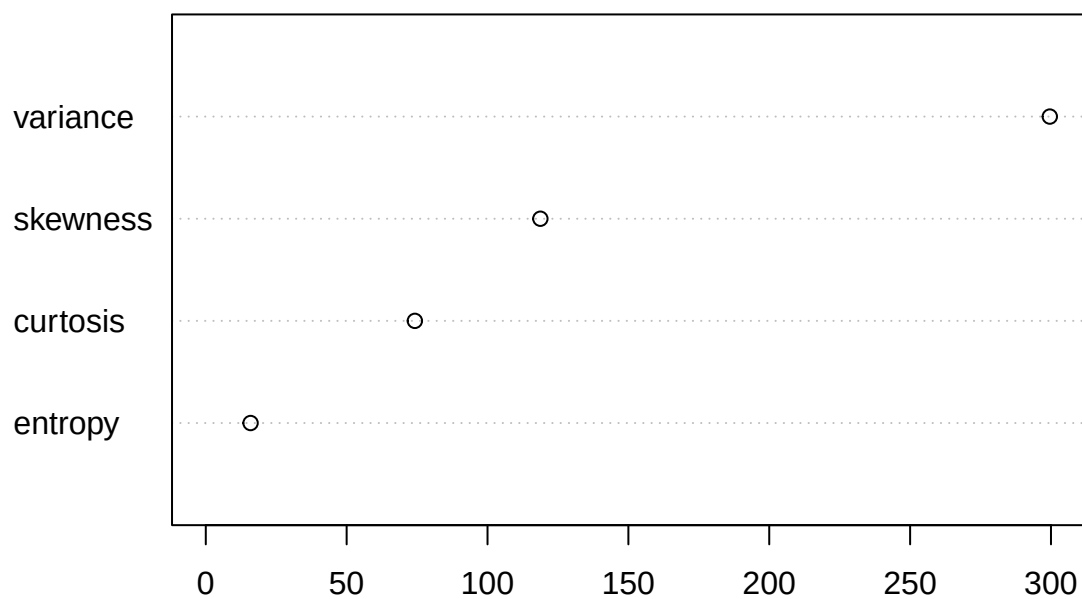
Mean Decrease Gini

```
varImp(final_model, type=2)
```

```
##           Overall
## variance 299.58446
## skewness 118.75781
## curtosis  74.21030
## entropy  15.89303
```

```
# Plot MeanDecreaseGini
varImpPlot(final_model, type=2)
```

final_model



plot

MeanDecreaseGini

Prediction

```
pred_caret <- predict(
  rf_caret,
  newdata = test.data
)
```

Confusion Matrix

```
confusionMatrix(
  pred_caret,
  test.data$class,
  positive = "1"
)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0    1
##           0 149    1
##           1   3 189
##
##           Accuracy : 0.9883
##           95% CI : (0.9703, 0.9968)
##           No Information Rate : 0.5556
```

```
##      P-Value [Acc > NIR] : <2e-16
##
##              Kappa : 0.9763
##
## Mcnemar's Test P-Value : 0.6171
##
##      Sensitivity : 0.9947
##      Specificity : 0.9803
##      Pos Pred Value : 0.9844
##      Neg Pred Value : 0.9933
##      Prevalence : 0.5556
##      Detection Rate : 0.5526
##      Detection Prevalence : 0.5614
##      Balanced Accuracy : 0.9875
##
##      'Positive' Class : 1
##
```

Regression Example

Import data

```
data("Boston", package = "MASS")
head(Boston)
```

```
##      crim zn indus chas   nox   rm age   dis rad tax ptratio  black lstat
## 1 0.00632 18  2.31    0 0.538 6.575 65.2 4.0900   1 296    15.3 396.90  4.98
## 2 0.02731  0  7.07    0 0.469 6.421 78.9 4.9671   2 242    17.8 396.90  9.14
## 3 0.02729  0  7.07    0 0.469 7.185 61.1 4.9671   2 242    17.8 392.83  4.03
## 4 0.03237  0  2.18    0 0.458 6.998 45.8 6.0622   3 222    18.7 394.63  2.94
## 5 0.06905  0  2.18    0 0.458 7.147 54.2 6.0622   3 222    18.7 396.90  5.33
## 6 0.02985  0  2.18    0 0.458 6.430 58.7 6.0622   3 222    18.7 394.12  5.21
##      medv
## 1 24.0
## 2 21.6
## 3 34.7
## 4 33.4
## 5 36.2
## 6 28.7
```

```
str(Boston)
```

```
## 'data.frame':    506 obs. of  14 variables:
## $ crim   : num  0.00632 0.02731 0.02729 0.03237 0.06905 ...
## $ zn     : num  18 0 0 0 0 12.5 12.5 12.5 12.5 ...
## $ indus  : num  2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
## $ chas   : int   0 0 0 0 0 0 0 0 0 0 ...
## $ nox    : num  0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
## $ rm     : num  6.58 6.42 7.18 7 7.15 ...
## $ age    : num  65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
```

```
## $ dis : num 4.09 4.97 4.97 6.06 6.06 ...
## $ rad : int 1 2 2 3 3 3 5 5 5 ...
## $ tax : num 296 242 242 222 222 222 311 311 311 311 ...
## $ ptratio: num 15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
## $ black : num 397 397 393 395 397 ...
## $ lstat : num 4.98 9.14 4.03 2.94 5.33 ...
## $ medv : num 24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
```

Remove Missing Value

```
Boston <- na.omit(Boston)
str(Boston)
```

```
## 'data.frame': 506 obs. of 14 variables:
## $ crim : num 0.00632 0.02731 0.02729 0.03237 0.06905 ...
## $ zn : num 18 0 0 0 0 0 12.5 12.5 12.5 12.5 ...
## $ indus : num 2.31 7.07 7.07 2.18 2.18 2.18 7.87 7.87 7.87 7.87 ...
## $ chas : int 0 0 0 0 0 0 0 0 0 0 ...
## $ nox : num 0.538 0.469 0.469 0.458 0.458 0.458 0.524 0.524 0.524 0.524 ...
## $ rm : num 6.58 6.42 7.18 7 7.15 ...
## $ age : num 65.2 78.9 61.1 45.8 54.2 58.7 66.6 96.1 100 85.9 ...
## $ dis : num 4.09 4.97 4.97 6.06 6.06 ...
## $ rad : int 1 2 2 3 3 3 5 5 5 5 ...
## $ tax : num 296 242 242 222 222 222 311 311 311 311 ...
## $ ptratio: num 15.3 17.8 17.8 18.7 18.7 18.7 15.2 15.2 15.2 15.2 ...
## $ black : num 397 397 393 395 397 ...
## $ lstat : num 4.98 9.14 4.03 2.94 5.33 ...
## $ medv : num 24 21.6 34.7 33.4 36.2 28.7 22.9 27.1 16.5 18.9 ...
```

There are no missing value in the dataset

Data Splitting

```
set.seed(123)
training.samples <- Boston$medv %>% createDataPartition(p=0.8, list=FALSE)
train.data <- Boston[training.samples,]
test.data <- Boston[-training.samples,]
```

Best Model

```
set.seed(123)

# 10-fold CV
control <- trainControl(
  method = "cv",
  number = 10
)
```

```
# train the rf using 10-fold CV
```

```
model <- train(  
  medv ~ .,  
  data = train.data,  
  method = "rf",  
  trControl = control  
)
```

```
print(model)
```

```
## Random Forest
```

```
##
```

```
## 407 samples
```

```
## 13 predictor
```

```
##
```

```
## No pre-processing
```

```
## Resampling: Cross-Validated (10 fold)
```

```
## Summary of sample sizes: 366, 367, 366, 366, 366, ...
```

```
## Resampling results across tuning parameters:
```

```
##
```

##	mtry	RMSE	Rsquared	MAE
##	2	3.635979	0.8577443	2.490950
##	7	3.200443	0.8828334	2.206899
##	13	3.342935	0.8705382	2.311711

```
##
```

```
## RMSE was used to select the optimal model using the smallest value.
```

```
## The final value used for the model was mtry = 7.
```

Access bestTune

```
print(model$bestTune)
```

```
## mtry
```

```
## 2 7
```

Prediction

```
pred <- predict(  
  model,  
  newdata = test.data  
)
```

RMSE

```
RMSE(pred, test.data$medv)
```

```
## [1] 3.107002
```

Model with optimal mtry and 500 tree

```
set.seed(123)
rf <- randomForest(
  medv~.,
  data=Boston,
  ntree=500,
  mtry=7,
  keep.forest=FALSE, #not save each individual CART tree in the output
  importance=TRUE
)
rf
```

```
##
## Call:
## randomForest(formula = medv ~ ., data = Boston, ntree = 500,      mtry = 7, keep.forest = FALSE, im
##               Type of random forest: regression
##               Number of trees: 500
## No. of variables tried at each split: 7
##
##               Mean of squared residuals: 9.876521
##               % Var explained: 88.3
```

RMSE

```
sqrt(rf$mse[500])
```

```
## [1] 3.142693
```

Variable Importance Values

```
importance(rf)
```

##		%IncMSE	IncNodePurity
##	crim	15.650276	2077.63508
##	zn	2.096193	74.01045
##	indus	9.908350	1550.15527
##	chas	1.834505	89.85939
##	nox	19.480394	2090.06065
##	rm	45.281438	14604.65858
##	age	14.099022	854.13476
##	dis	21.446882	2379.79218
##	rad	6.842151	197.82982
##	tax	11.167872	651.73447
##	ptratio	15.750284	1678.73412
##	black	9.272033	538.92145
##	lstat	36.673347	15387.09079

```
varImpPlot(rf)
```

rf

